R basics

Ling Yang

Supercomputer Center, Westlake University.

2019/09/06

Overview and History

- R is a dialect of the S language.
- S was initiated in 1976 as a statistical analysis environment.

In "Stages in the Evolution of S", John Chambers writes:

> *"[W]e wanted users to be able to begin in an interactive environment, where they did not consciously think of themselves as programming. Then as their needs became clearer and their sophistication increased, they should be able to slide gradually into programming, when the language and system aspects would become more important."*

Features of R

- Quite lean; functionality is divided into modular packages.
- Graphics capabilities very sophisticated and better than most stat tools.
- Useful for interactive work, but contains a powerful programming language for developing new tools.
- Very active user community, especially Stack Overflow.
- In my experience, the convenience of *data processing* in R is the best of the best.

It's free!

Drawbacks of R

- Essentially based on very old technology.
- Sophisticated support for dynamic or 3D graphics.
- Objects generally stored in physical memory.

Design of the R system

The "base" R system and everything else.

- You can do everything by r-base downloaded from CRAN. It is an inferno!
- You can do something by packages. A huge network!
- There are 14579 packages on CRAN and 1741 packages on Bioconductor (2019-07-22)

Let's begin!

https://rextester.com/l/r_online_compiler

Objects

R has five basic classes of objects:

- character, e.g. “a”, “ABc”
- numeric (real numbers), e.g. 1, 2, 1.4142
- integer
- complex, e.g. 2i, 1+3i
- logical (TRUE/FALSE)

The most of objects are vectors, only containing same class.
BUT: *list* represents a vector can contain objects of different classes.

Attributes

R objects **CAN** have attributes:

- names, dimnames
- dimensions
- class
- length
- other user-defined attributes/metadata

Attributes of an object can be accessed using the attributes() function.

Numbers

- Numbers in R were generally treated as numeric objects. (double precision real numbers)
- If you explicitly want an integer, you need to specify the L suffix. e.g. 1L
- Inf represents infinity. e.g. 1 / 0, 1 / Inf
- NaN represents an undefined value ("not a number"); NA represents missing values ("Not Available"); NULL represents the null object in R.

Input

At the R prompt, we type *expressions*. The <- symbol is the assignment operator.

```
> x <- 1
> print(x)
```

```
## [1] 1
```

```
> x
```

```
## [1] 1
```

```
> msg <- "hello"
```

Evaluation

When a complete expression is entered at the prompt, it is *evaluated* and the result of the evaluated expression is returned. The result may be *auto-printed*.

```
> x <- 5 # nothing printed
> x # auto-printing occurs
```

```
## [1] 5
```

```
> print(x) # explicit printing
```

```
## [1] 5
```

The # character indicates a *comment*. Anything to the right of the # is ignored. The [1] indicates that x is a vector and 5 is the first element.

Printing

```
> x <- 1:20
> x
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
## [16] 16 17 18 19 20
```

The : operator is used to create integer sequences.

Creating Vectors

The c() function can be used to create vectors of objects.

```
> x <- c(0.5,0.6) # numeric
> x <- c(T,FALSE) # logical
> x <- c("a","b","c") # character
> x <- 10:5 # integer
> # Using the vector() function
> x <- vector("numeric",length=10)
> x

##  [1] 0 0 0 0 0 0 0 0 0 0
```

Mixing Objects

What about the following?

```
> y <- c(1,7,"a")
> class(y)
```

```
## [1] "character"
```

```
> y <- c(T,2)
> class(y)
```

```
## [1] "numeric"
```

When different objects are mixed in a vector, **coercion** occurs so that every element in the vector is of the same class.

Explicit Coercion

Objects can be explicitly coerced from one class to another using the as.* functions, if available.

```
> x <- 0:6
> as.numeric(x)
```

```
## [1] 0 1 2 3 4 5 6
```

```
> as.logical(x)
```

```
## [1] FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
> as.character(x)
```

```
## [1] "0" "1" "2" "3" "4" "5" "6"
```

Explicit Coercion (cont'd)

Nonsensical coercion results in NAs.

```
> x <- c("a","b","c")
> as.numeric(x)

## [1] NA NA NA

> as.logical(x)

## [1] NA NA NA
```

Matrices

Matrices are vectors with a *dimension* attribute. The dimension attribute is itself an integer vector of length 2 (nrow, ncol).

```
> m <- matrix(nrow=2,ncol=3)
> dim(m)
```

```
## [1] 2 3
```

```
> attributes(m)
```

```
## $dim
## [1] 2 3
```

Matrices (cont'd)

Matrices are constructed **column-wise**, so entries can be thought of starting in the "upper left" corner and running down the columns.

```
> m <- matrix(1:6,nrow=2,ncol=3)
> m
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

Matrices (cont'd)

Matrices can also be created directly from vectors by adding a dimension attribute.

```
> m <- 1:10
> dim(m) <- c(2,5)
> m
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

cbind-ing and rbind-ding

Matrices can be created by **column-biding** or **row-binding** with cbind() and rbind().

```
> x <- 1:3
> y <- 10:12
> cbind(x,y)
```

```
##      x  y
## [1,] 1 10
## [2,] 2 11
## [3,] 3 12
```

```
> rbind(x,y)
```

```
##   [,1] [,2] [,3]
## x    1    2    3
## y   10   11   12
```

Lists

Lists are a special type of vectors that can contain elements of different classes.

```
> x <- list(1,"a",1 + 4i)
> x
```

```
## [[1]]
## [1] 1
##
## [[2]]
## [1] "a"
##
## [[3]]
## [1] 1+4i
```

Factors

Factors are used to represent categorical data. It can be unordered or ordered. **One can think of a factor as an integer vector where each integer has a label.**

- Factors are treated specially by modelling functions like lm() and glm().
- Using factors with labels is **better** than using integers because factors are self-describing; having a variable that has values "Male" and "Female" is better than a variable that has value 1 and 2.

Factors (cont'd)

```
> x <- factor(c("sample_a","sample_a","sample_n"))
> x

## [1] sample_a sample_a sample_n
## Levels: sample_a sample_n

> table(x)

## x
## sample_a sample_n
##        2        1

> unclass(x)

## [1] 1 1 2
## attr(,"levels")
## [1] "sample_a" "sample_n"
```

Factors (cont'd)

The order of the levels can be set using the levels argument to factor(). This can be **very important** in linear modelling because the first level is used as the baseline level.

```
> x <- factor(c("sample_a","sample_a","sample_n",
+               "sample_a"),levels=c("sample_n",
+                                    "sample_a"))
> x

## [1] sample_a sample_a sample_n sample_a
## Levels: sample_n sample_a
```

Missing Values

Missing values are denoted by NA or NaN for undefined mathematical operation.

- is.na() is used to test objects if they are NA.
- is.nan() is used to test for NaN.
- NA values have a class also, so there are integer NA, character NA, etc.
- A NaN value is also NA.

Missing Values (cont'd)

```
> x <- c(1,2,NA,10,3)
> is.na(x)

## [1] FALSE FALSE  TRUE FALSE FALSE

> is.nan(x)

## [1] FALSE FALSE FALSE FALSE FALSE

> x <- c(1,2,NaN,NA,4)
> is.na(x)

## [1] FALSE FALSE  TRUE  TRUE FALSE

> is.nan(x)

## [1] FALSE FALSE  TRUE FALSE FALSE
```

Data Frames

Data frames are used to store tabular data

- They are represented as a special type of list where every element of the list has to have the same length.
- Each element of the list can be thought of as a column and the length of each element of the list is the number of rows.
- Unlike matices, data frames can store **different classes** of objects in each column (just like lists); matrices must have every element be the same class.
- Data frames also have a special attribute called row.names.
- Data frames are usually created by calling read.table() or read.csv().

Data Frames (cont'd)

```
> x <- data.frame(foo=1:4,bar=c(T,T,F,F))
> x
```

```
##   foo   bar
## 1   1  TRUE
## 2   2  TRUE
## 3   3 FALSE
## 4   4 FALSE
```

```
> nrow(x)
```

```
## [1] 4
```

```
> ncol(x)
```

```
## [1] 2
```

Names

R objects can also have names, which is very useful for writing readable code and plotting.

```
> x <- 1:3
> names(x)
```

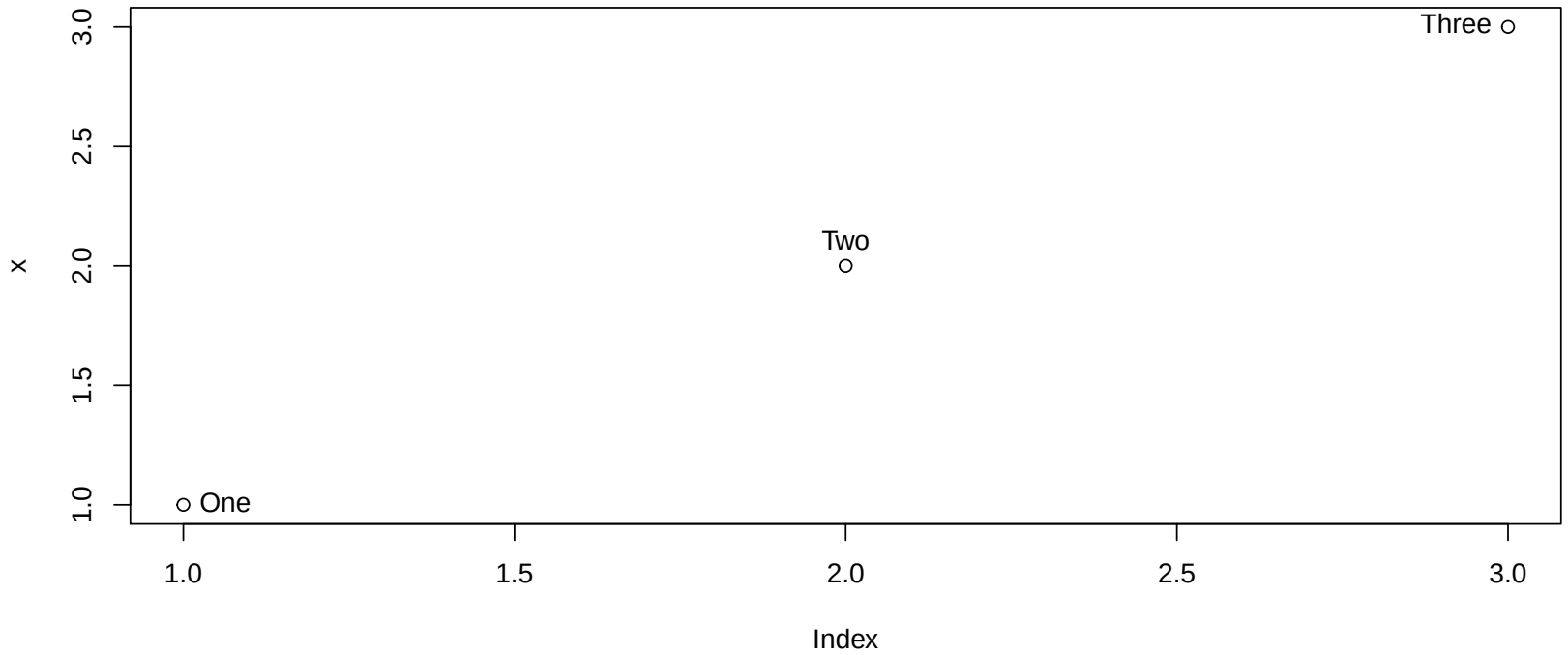
```
## NULL
```

```
> names(x) <- c("One","Two","Three")
> x
```

```
##   One   Two Three
##     1     2     3
```

Names (cont'd)

```
> plot(x)
> text(x,labels=names(x),pos=c(4,3,2))
```



Subsetting

Subsetting

There are a number of operators that can be used to extract subsets of R objects.

- [always returns an object of the same class as the original; can be used to select more than one element.
- [[is used to extract elements of a list or a data frame; it can only be used to extract a single element and the class of the returned object will not necessarily be a list or data frame.
- $ is used to extract elements of a list or data frame by name; semantics are similar to hat of [[.

Subsetting (cont'd)

```
> x <- c("a","b","c","c","d","a")
> x[1]

## [1] "a"

> x[1:4]

## [1] "a" "b" "c" "c"

> u <- x > "a"
> u

## [1] FALSE  TRUE  TRUE  TRUE  TRUE FALSE

> x[u]

## [1] "b" "c" "c" "d"
```

Subsetting a Matrix

Matrices can be subsetted in the usual way with (i,j) type indices.

```
> x <- matrix(1:6,2,3)
> x[1,2]
```

```
## [1] 3
```

```
> x[2,1]
```

```
## [1] 2
```

Indices can also be missing.

```
> x[1,]
```

```
## [1] 1 3 5
```

Subsetting a Matrix (cont'd)

By default, when a single element of a matrix is retrieved, it is returned as a vector of length 1 rather than a 1×1 matrix. This behavior can be turned off by setting drop = FALSE.

```
> x <- matrix(1:6,2,3)
> x[1,2]
```

```
## [1] 3
```

```
> x[1,2,drop=F]
```

```
##      [,1]
## [1,]    3
```

Subsetting a Matrix (cont'd)

Similarly, subsetting a single column or a single row will give you a vector, not a matrix (by default).

```
> x <- matrix(1:6,2,3)
> x[1,]
```

```
## [1] 1 3 5
```

```
> x[1,,drop=F]
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
```

Subsetting Lists

```
> x <- list(foo=1:4,bar=0.6)
> x[1] # the class is a list

## $foo
## [1] 1 2 3 4

> x[[1]] # the class is a integer

## [1] 1 2 3 4

> x[["bar"]]

## [1] 0.6

> x["bar"]

## $bar
## [1] 0.6
```

Subsetting Lists (cont'd)

Extracting multiple elements of a list.

```
> x <- list(foo=1:4,bar=0.6,baz="hello")
> x[c(1,3)]
```

```
## $foo
## [1] 1 2 3 4
##
## $baz
## [1] "hello"
```

Subsetting Lists (cont'd)

The [[operator can be used with *computed* indices; $ can only be used with literal names.

```
> x <- list(foo=1:4,bar=0.6,baz="hello")
> name <- "foo"
> x[[name]] # computed index for 'foo'

## [1] 1 2 3 4

> x$name # element 'name' doesn't exist!

## NULL

> x$foo # element 'foo' does exist

## [1] 1 2 3 4
```

Subsetting Nested Elements of a List

The [[can take an integer sequence.

```
> x <- list(a=list(10,12,14),b=c(3.14,2.81))
> x[[c(1,3)]]

## [1] 14

> x[[1]][[3]]

## [1] 14

> x[[c(2,1)]]

## [1] 3.14
```

Partial Matching

Partial matching of names is allowed with [[and $.

```
> x <- list(aardvark=1:5)
> x$a
```

```
## [1] 1 2 3 4 5
```

```
> x[["a"]]
```

```
## NULL
```

```
> x[["a",exact=F]]
```

```
## [1] 1 2 3 4 5
```

Removing NA Values

A common task is to remove missing values (NAs).

```
> x <- c(1,2,NA,4,NA,5)
> bad <- is.na(x)
> x[!bad]
```

```
## [1] 1 2 4 5
```

Removing NA Values (cont'd)

What if there are multiple things and you want to take the subset with no missing values?

```
> x <- c(1:3,NA,5,NA,7)
> y <- c(letters[1:3],NA,"e",NA,"g")
> good <- complete.cases(x,y)
> good
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE  TRUE
```

```
> x[good]
```

```
## [1] 1 2 3 5 7
```

```
> y[good]
```

```
## [1] "a" "b" "c" "e" "g"
```

Removing NA Values (cont'd)

```
> airquality[4:7,]
```

```
##   Ozone Solar.R Wind Temp Month Day
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
## 7    23     299  8.6   65     5   7
```

```
> good <- complete.cases(airquality)
> airquality[good,][4:7,]
```

```
##   Ozone Solar.R Wind Temp Month Day
## 4    18     313 11.5   62     5   4
## 7    23     299  8.6   65     5   7
## 8    19      99 13.8   59     5   8
## 9     8      19 20.1   61     5   9
```

Vectorized Operations

Vectorized Operations

Many operations in R are *vectorized* making code more efficient, concise, and easier to read.

```
> x <- 1:4; y <- 6:9
> x + y
```

```
## [1]  7  9 11 13
```

```
> x > 2
```

```
## [1] FALSE FALSE  TRUE  TRUE
```

```
> x >= 2
```

```
## [1] FALSE  TRUE  TRUE  TRUE
```

```
> y == 8
```

```
## [1] FALSE FALSE  TRUE FALSE
```

Vectorized Matrix Operations

```
> x <- matrix(1:4,2,2); y <- matrix(rep(10,4),2,2)
> x * y
```

```
##      [,1] [,2]
## [1,]   10   30
## [2,]   20   40
```

```
> x / y
```

```
##      [,1] [,2]
## [1,]  0.1  0.3
## [2,]  0.2  0.4
```

```
> x %*% y # true matrix multiplication
```

```
##      [,1] [,2]
## [1,]   40   40
## [2,]   60   60
```

Reading and Writing Data

Reading Data

There are a few principal functions reading data into R.

- read.table, read.csv, read.delim, for reading tabular data
- readLines, for reading lines of a text file
- source, for reading in R code files (inverse of dump)
- dget, for reading in R code files (inverse of dput)
- load, for reading in saved workspaces
- unserialize, for reading single R objects in binary form

Writing Data

There are analogous functions for writing data to files

- write.table
- writeLines
- dump
- dput
- save
- serialize

Reading Data Files with read.table

The read.table function is one of the most commonly used functions for reading data. It has a few important arguments:

- **file**, the name of a file, or a connection
- **header**, logical indicating if the file has a header line
- **sep**, a string indicating how the columns are separated
- **colClasses**, a character vector indicating the class of each column in the dataset
- **nrows**, the number of rows in the dataset
- **comment.char**, a character string indicating the comment character
- **skip**, them number o lines to skip from the beginning
- **stringsAsFactors**, should character variables be coded as factors?

read.table

For small to moderately sized datasets, you can usually call read.table without specifying any other arguments

```
data <- read.table("foo.txt")
```

R will automatically

- skip lines that begin with a #
- figure out how many rows there are (and how much memory needs to be allocated)
- figure what type of variable is in each column of the table

Telling R all these things directly makes R run faster and more efficiently.

- read.csv is identical to read.table except that the default separator is a comma.

With much larger datasets, data.table::fread is a much better alternative.

Summary

- Data Types
- Subsetting
- Vectorized Operations
- Reading and Writing Data

Introduction to Supercomputer Center

Supercomputer Center

hpc@wesklake.edu.cn

- 170 nodes
- 1.44 PB storage
- 4080 cores
- 140.25 TFlops

Thanks very much!